PSL INTEGRITY SHIELD

Sprint Guide

Anti-Match-Fixing Transparency Protocol for Pakistan Super League

---

Entangled Hackathon 2026 · WireFluid Network (Chain 92533)

⚠ **DEADLINE: April 15, 2026 at 12:00 PM (Noon)** ⚠

Team

Zayan Ahmed · Mehroz Muneer · Abdul Rehman · Mufeez · Uzair

Coordinated by Mahad

1. Critical Hackathon Info

⚠ **HARD DEADLINE: Wednesday, April 15, 2026 at 12:00 PM (Noon)**

1.1 Submission Requirements

Email to submissions@wirefluid.com from the same email used to register. Include:

1. Source Code / Public GitHub Repository link
2. A working Demo Link of the dApp
3. A Presentation (PDF/Slides) explaining the project, real-world impact, and WireFluid usage
4. ALL relevant transaction hashes proving deployment on WireFluid Testnet

1.2 PSL 3x Multiplier

PSL-related projects earn a 3x scoring multiplier. This must be real-world utility, not cosmetic theming. Our Integrity Shield directly addresses match-fixing — Pakistan cricket's biggest credibility problem — which qualifies fully.

1.3 Network Configuration

Setting	Value
Network Name	WireFluid Testnet
RPC URL	https://evm.wirefluid.com
Chain ID	92533
Currency	WIRE
Block Explorer	https://scan.wirefluid.com
Faucet	https://faucet.wirefluid.com
Developer Docs	https://docs.wirefluid.com/developer-guide

1.4 Post on X

Use #BuildOnWireFluid on X (Twitter) with transaction hashes as you build. This is expected by the organizers.

2. Team Structure — 3 Working Groups

The team is split into three groups. Each group owns a clear vertical with defined handoff points. Work runs in parallel.

Group	Members	Primary Ownership	Delivers
Group A	Mufeez + Abdul Rehman	All 3 smart contracts, Hardhat config, deploy script, testing, testnet deployment	3 compiled + tested .sol files, deployed contract addresses, all tx hashes
Group B	Zayan + Mehroz	Entire frontend dApp (index.html), MetaMask integration, all UI sections, ethers.js wiring	Single-page dApp with full contract interaction
Group C	Uzair	README, presentation PDF, demo video, submission email, GitHub repo setup, #BuildOnWireFluid posts	Submission-ready package sent before noon

2.1 Group A: Mufeez + Abdul Rehman — Contracts + Testing

Group A owns everything Solidity. Both members work on contracts together, splitting the three contracts between them, pair-reviewing each other's code, and running tests continuously. This is the foundation — nothing else works until contracts compile and deploy.

Contract Split:

Mufeez: MatchOracle.sol + PredictionEngine.sol

Abdul Rehman: AnomalyTracker.sol + test suite (IntegrityShield.test.js)

Both review each other's contracts before pushing. Abdul Rehman writes tests for all 3 contracts as they're written.

MatchOracle.sol (Mufeez) — ~150–180 lines:

- Match creation: createMatch(matchId, team1, team2) — owner only
- Reporter registration: registerReporter() payable, msg.value >= 0.01 ether
- Event submission: submitEvent(matchId, eventHash) — only registered reporters, only during Live state
- State machine: enum MatchState { Upcoming, Live, Completed } with startMatch() and endMatch()
- Slashing: slashReporter(address) — owner can slash, stake goes to contract
- View function: getMatchState(matchId) — used by PredictionEngine and AnomalyTracker

- Imports: OpenZeppelin v5 Ownable + ReentrancyGuard
- Events: MatchCreated, ReporterRegistered, EventSubmitted, MatchStarted, MatchEnded, ReporterSlashed

PredictionEngine.sol (Mufeez) — ~140–170 lines:

- Constructor takes MatchOracle address
- commitPrediction(matchId, hash) — match must be Upcoming
- revealPrediction(matchId, winner, totalRuns, salt) — match must be Completed
- Reveal validates: keccak256(abi.encodePacked(winner, totalRuns, salt)) must equal stored commit
- Aggregation: track total reveals, winner counts per team, sum of predicted runs
- Events: PredictionCommitted, PredictionRevealed, ResultsAggregated

AnomalyTracker.sol (Abdul Rehman) — ~160–200 lines:

- Constructor takes MatchOracle address
- raiseFlag(matchId, reason) payable — match must be Completed
- voteOnFlag(flagId, support) payable — stake-weighted voting
- resolveFlag(flagId) — quorum check (3+ votes), if votesFor > votesAgainst: upheld (return flagger stake, integrity score drops 10), else: rejected (slash flagger stake)
- getIntegrityScore(matchId) — starts at 100, drops per upheld flag
- Events: FlagRaised, VoteCast, FlagResolved, IntegrityScoreUpdated

Test Suite (Abdul Rehman):

```
const { expect } = require('chai');
const { ethers } = require('hardhat');

describe('PSL Integrity Shield', function () {
  let matchOracle, predictionEngine, anomalyTracker;
  let owner, reporter, fan1, fan2;

  beforeEach(async function () {
    [owner, reporter, fan1, fan2] = await ethers.getSigners();
    const MO = await ethers.getContractFactory('MatchOracle');
    matchOracle = await MO.deploy();
    const PE = await ethers.getContractFactory('PredictionEngine');
    predictionEngine = await PE.deploy(matchOracle.target);
    const AT = await ethers.getContractFactory('AnomalyTracker');
    anomalyTracker = await AT.deploy(matchOracle.target);
  });

  describe('MatchOracle', () => {
    it('should create a match');
    it('should register reporter with correct stake');
```

```
    it('should reject insufficient stake');
    it('should submit events only during Live');
    it('should enforce state transitions');
    it('should slash reporter');
  });

  describe('PredictionEngine', () => {
    it('should accept commit before match');
    it('should reject commit after match starts');
    it('should validate reveal hash');
    it('should reject wrong salt');
  });

  describe('AnomalyTracker', () => {
    it('should raise flag with stake');
    it('should accept votes');
    it('should resolve upheld flag correctly');
    it('should slash rejected flag stake');
    it('should update integrity score');
  });
});
```

Deploy Script (Both):

- Deploy MatchOracle first, then PredictionEngine(matchOracle.target), then AnomalyTracker(matchOracle.target)
- Create 1–2 sample matches, register a reporter, submit sample events
- Print all addresses and tx hashes — share immediately with Group B and Group C

Handoff to Group B:

As soon as contracts compile, **share the ABI JSON files** (from artifacts/contracts/*.json) with Group B so they can wire the frontend. After testnet deployment, share the 3 contract addresses. Don't wait until everything is perfect — Group B needs ABIs early to build in parallel.

2.2 Group B: Zayan + Mehroz — Frontend dApp

Group B owns the entire frontend. Single file: frontend/index.html. No build tools, no React — HTML + CSS + vanilla JS + ethers.js v6 from CDN. Split the UI by sections so both can work without stepping on each other.

Section Ownership:

Zayan: Navigation + MetaMask connect, dashboard (match cards grid), reporter panel (registration + stake display), audit trail viewer

Mehroz: Match detail page (event feed + status), prediction panel (commit form + reveal form + salt management), anomaly section (flag form + voting buttons + progress bars + integrity badge)

HTML Structure (Both Follow):

```
<html>
<head>
  <title>PSL Integrity Shield</title>
  <style>/* dark theme CSS */</style>
</head>
<body>
  <!-- === ZAYAN === -->
  <nav><!-- wallet connect, network status, address display --></nav>
  <div id="dashboard"><!-- match cards grid --></div>
  <div id="reporter-panel"><!-- register, active reporters --></div>
  <div id="audit-trail"><!-- chronological event logs --></div>

  <!-- === MEHROZ === -->
  <div id="match-detail"><!-- event timeline, match status --></div>
  <div id="prediction-panel"><!-- commit/reveal forms --></div>
  <div id="anomaly-section"><!-- flags, voting, integrity score --></div>

  <script
src="https://cdnjs.cloudflare.com/ajax/libs/ethers/6.9.0/ethers.umd.min.js"></script>
  <script>
    const CONFIG = {
      MATCH_ORACLE: '0x...',       // from Group A after deploy
      PREDICTION_ENGINE: '0x...',
      ANOMALY_TRACKER: '0x...',
      CHAIN_ID: 92533,
      RPC_URL: 'https://evm.wirefluid.com',
      EXPLORER: 'https://scan.wirefluid.com'
    };
    // === ABI objects pasted here after Group A compiles ===

    // === ZAYAN's functions ===
    // connectWallet(), loadDashboard(), registerReporter(), loadAuditTrail()
```

```
    // === MEHROZ's functions ===
    // loadMatchDetail(), commitPrediction(), revealPrediction(),
    // raiseFlag(), voteOnFlag(), loadIntegrityScore()
  </script>
</body>
</html>
```

Zayan's Sections in Detail:

- MetaMask connect: handle 3 states — not installed (show install link), wrong network (auto-switch to chain 92533 via wallet_addEthereumChain), connected (show truncated address + dashboard)
- Dashboard: grid of match cards, each showing matchId, team1 vs team2, status badge (Upcoming=blue, Live=green, Completed=gray), integrity score badge, click to open match detail
- Reporter panel: register button with stake display (0.01 WIRE), list of registered reporters from contract events
- Audit trail: pull all events from all 3 contracts using ethers.js queryFilter, display chronologically with tx hash links to https://scan.wirefluid.com/tx/{hash}

Mehroz's Sections in Detail:

- Match detail: selected match info, list of submitted event hashes (from MatchOracle events), current state
- Prediction commit form: winner dropdown (team1/team2), total runs input, auto-generate salt button (random bytes32), display salt with copy button, store salt in localStorage as backup
- Prediction reveal form: pre-fill salt from localStorage, winner + runs fields, submit reveal, show success/failure
- Flag form: reason textarea + stake input, raise flag button
- Voting: support/oppose buttons per active flag, progress bars showing vote weight percentages
- Integrity badge: green (#00b894) for 80–100, yellow (#fdcb6e) for 50–79, red (#e74c3c) for <50

Salt Management (Mehroz — Critical):

If users lose their salt, they cannot reveal. The frontend must auto-generate salt, display it prominently, store in localStorage keyed by matchId, and pre-fill on the reveal form. Add a warning message telling users to save their salt.

Design Specs (Both Follow):

- Background: #1a1a2e, card backgrounds: #16213e
- Accent green: #00b894, accent red: #e74c3c
- Text: #ffffff primary, #b2bec3 secondary

- All blockchain calls show loading spinners
- Failed transactions show user-friendly error messages

Merge Strategy:

Each person works inside their marked comment blocks (=== ZAYAN === and === MEHROZ ===). Never edit the other person's section. When merging, if conflicts appear, the person who caused them resolves them.

2.3 Group C: Uzair — Submission Package

Uzair owns everything the judges see first: README, slides, demo video, and the submission email itself. He also sets up the GitHub repo and manages the #BuildOnWireFluid posts.

Deliverables:

1. GitHub repo setup: create public repo, add .gitignore (node_modules, .env, artifacts, cache), initial commit with folder structure, share repo link with everyone
2. README.md: problem statement (match-fixing in Pakistan cricket), solution overview (3 interlocking contracts), architecture diagram (ASCII art from the existing README), quick start guide (install, compile, test, deploy), judging criteria coverage table, contract addresses + tx hashes (filled after Group A deploys)
3. Presentation PDF (5–7 slides): Slide 1: Problem (match-fixing stats + trust erosion), Slide 2: Solution (3-contract system), Slide 3: Architecture diagram, Slide 4: Demo screenshots (from Group B's frontend), Slide 5: Real-world impact (why blockchain is essential for this), Slide 6: Team
4. Demo video (2–3 minutes): connect wallet → create match → register reporter → submit events → commit prediction → reveal → raise flag → vote → check integrity score — screen record the frontend flow end-to-end
5. Submission email: to submissions@wirefluid.com, from registration email, include GitHub link + demo link + presentation PDF + all tx hashes
6. #BuildOnWireFluid posts on X: post progress screenshots + tx hashes throughout the sprint

Demo Link Options:

For the working demo link required in submission, use GitHub Pages:

```
# Repo Settings > Pages > Source: main branch, /frontend folder
# Demo URL: https://<username>.github.io/psl-integrity-shield/
```

What Uzair Needs From Others:

- From Group A: contract addresses, all tx hashes, test results screenshot, gas report
- From Group B: frontend screenshots (dashboard, match detail, voting UI, integrity badge), working demo URL
- From everyone: any notable wins or features worth highlighting in the presentation

3. Sprint Timeline

⚠ **Work backwards from April 15 at 12:00 PM noon.**

Phase	Time Block	Tasks	Who
Setup	First 1–2 hours	Uzair creates GitHub repo + folder structure. Everyone clones, installs deps, configures .env with private keys and test WIRE from faucet	All
Build	Next 5–7 hours	Group A: write all 3 contracts + tests | Group B: build frontend shell + MetaMask connect + dashboard | Uzair: draft README + start presentation slides	Parallel
Integration	Next 3–4 hours	Group A: deploy to testnet, share ABIs + addresses | Group B: paste ABIs + addresses, wire all UI to live contracts | Uzair: collect tx hashes + screenshots	Parallel
Test + Fix	Next 2–3 hours	Full end-to-end testing in browser against testnet. Fix bugs. Group B polishes UI. Group A runs final test suite.	All
Submit	Final 1–2 hours	Uzair: finalize README with addresses, finish slides, record demo video, send submission email. Everyone reviews before send.	All

3.1 Handoff Points

Group A → Group B: Share ABI JSON files (artifacts/contracts/*.json) as soon as contracts compile. Don't wait for tests. After testnet deploy, share the 3 contract addresses immediately.

Group A → Uzair: Share deployed contract addresses + all tx hashes after deployment. Share test results screenshot.

Group B → Uzair: Share frontend screenshots throughout. Once frontend works end-to-end, share the GitHub Pages demo URL.

Uzair → Everyone: Share GitHub repo link immediately. Keep README updated as information comes in. Final review request 1 hour before deadline.

4. Setup & Dependencies

4.1 Prerequisites

- Node.js v18+ (node -v to check)
- MetaMask browser extension installed
- Git installed and configured
- VS Code recommended
- Test WIRE tokens from https://faucet.wirefluid.com

4.2 Project Initialization

```
mkdir psl-integrity-shield && cd psl-integrity-shield
npm init -y
npx hardhat init     # Choose: Create a JavaScript project
```

```
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npm install @openzeppelin/contracts dotenv
```

```
mkdir -p contracts scripts test frontend
echo 'PRIVATE_KEY=your_wallet_private_key_here' > .env
```

4.3 hardhat.config.js

```
require("@nomicfoundation/hardhat-toolbox");
require("dotenv").config();

module.exports = {
  solidity: "0.8.20",
  networks: {
    wirefluid: {
      url: "https://evm.wirefluid.com",
      chainId: 92533,
      accounts: [process.env.PRIVATE_KEY],
    },
  },
};
```

4.4 Package Reference

Package	Version	Install	Purpose
hardhat	^2.19.0	npm i -D hardhat	Contract framework
@openzeppelin/contracts	^5.0.0	npm i @openzeppelin/contracts	Ownable, ReentrancyGuard

hardhat-toolbox	^4.0.0	npm i -D @nomicfoundation/hardhat-toolbox	Testing + compile
dotenv	^16.3.0	npm i dotenv	Load .env variables
ethers (frontend)	v6.9.0	CDN in index.html	Browser blockchain calls

5. Smart Contract Specifications

Reference for all groups. Group A builds from these specs. Group B uses them to know which functions to call. Uzair references them for the presentation.

5.1 MatchOracle.sol

State:

```
enum MatchState { Upcoming, Live, Completed }

struct Match {
    string team1;
    string team2;
    MatchState state;
    uint256 eventCount;
    uint256 createdAt;
}

uint256 public reporterStake = 0.01 ether;
mapping(uint256 => Match) public matches;
mapping(address => bool) public registeredReporters;
mapping(address => uint256) public reporterStakes;
```

Functions:

```
createMatch(uint256 matchId, string team1, string team2)  // onlyOwner
registerReporter()                                        // payable
submitEvent(uint256 matchId, bytes32 eventHash)           // onlyReporter, onlyLive
startMatch(uint256 matchId)                               // onlyOwner
endMatch(uint256 matchId)                                 // onlyOwner
slashReporter(address reporter)                           // onlyOwner
getMatchState(uint256 matchId) returns (MatchState)       // view
```

5.2 PredictionEngine.sol

Commit-Reveal Logic:

```
// COMMIT (before match starts):
// Frontend: salt = random bytes32
// Frontend: hash = keccak256(abi.encodePacked(winner, totalRuns, salt))
// User calls: commitPrediction(matchId, hash)

// REVEAL (after match ends):
// User calls: revealPrediction(matchId, winner, totalRuns, salt)
// Contract recomputes hash and validates against stored commit
```

Functions:

```
constructor(address _matchOracle)
commitPrediction(uint256 matchId, bytes32 hash)
revealPrediction(uint256 matchId, string winner, uint256 runs, bytes32 salt)
getAggregatedResults(uint256 matchId) returns (...)
```

5.3 AnomalyTracker.sol

Flag Lifecycle:

```
raiseFlag(matchId, reason)   // payable, match must be Completed
voteOnFlag(flagId, support)  // payable, stake-weighted
resolveFlag(flagId)          // quorum: 3+ votes
  --> Upheld: return flagger stake, integrity score drops 10
  --> Rejected: slash flagger stake
getIntegrityScore(matchId)   // starts 100, drops per upheld flag
```

6. Deployment & Submission Checklist

6.1 Deploy

```
npx hardhat run scripts/deploy.js --network wirefluid
```

After deployment:

1. Copy 3 contract addresses into frontend CONFIG object
2. Copy all tx hashes into README
3. Share addresses + hashes with Uzair immediately
4. Test frontend against live testnet contracts
5. Post tx hashes on X with #BuildOnWireFluid

6.2 Verification

```
npx hardhat verify --network wirefluid <CONTRACT_ADDRESS> <constructor_args>
```

If explorer doesn't support Hardhat verify, submit source manually.

6.3 Final Checklist

⚠ **Email to: submissions@wirefluid.com (from registration email)**

1. Public GitHub repository with all source code
2. Working demo link (GitHub Pages)
3. Presentation PDF (5–7 slides)
4. ALL transaction hashes: contract deployments + sample interactions
5. Demo video (2–3 minutes)

6.4 Commands Quick Reference

```
npx hardhat compile                                    # Compile
npx hardhat test                                       # Test
REPORT_GAS=true npx hardhat test                       # Gas report
npx hardhat run scripts/deploy.js --network wirefluid  # Deploy
npx hardhat verify --network wirefluid <ADDR> <ARGS>   # Verify
npx hardhat clean                                      # Clean
```

Ship it before noon. Good luck.